Python lookup index

Where to turn when you get stuck

Current course

Variables and input 3 Conditions 4 For and range 5 While 6 Functions and None 7 Quick check 8

Future reference

Strings 9 Dictionaries 10 Function patterns 11 Errors 12 Files 13 Classes 14 Testing 15

Search formula

Search Python plus your goal plus the value type. Example: Python remove item from list.

Ask a strong question

Share what you expected, what happened, the smallest code that reproduces it, and the full error message.

Keep learning

Read examples actively: predict, run, change one thing, and explain the new result.

Python Field Guide

A black and white lookup guide for learning Python

How to use this guide

Find the idea you need, copy the pattern, change the names and values, then run a small test.

Looking things up is part of programming

You do not need to memorize every command. You do need to explain what your code is doing.

Start with what we have learned

Pages 2 through 8 review our current course. Pages 9 through 16 prepare you for future lessons.

What we have learned

Current course reference

Values and variables

Store text, numbers, and True or False values under useful names.

Program decisions

Use if, elif, else, comparisons, and logic to choose which code runs.

Repetition

`Use for with a collection or range. Use while when repetition depends on a condition.`

Reusable code

Use functions, parameters, arguments, and return values to organize a program.

Core habit

Predict the result, run the code, compare, and investigate any difference.

Future testing and design

Build reliable programs in small steps

Arrange act assert

Arrange inputs, call the function, then check the result you expected.

Simple self check

assert power(7, 5) == 12
assert double(0) == 0

Test edges

Try empty lists, zero, negative numbers, missing values, and the smallest or largest allowed input.

Decompose

Break a large problem into named functions. Test each function before joining them together.

Comments

Explain why a decision exists. Let clear names and small functions explain what the code does.

Future classes and objects

Bundle related data and behavior

Class pattern

```
class Player:
def __init__(self, name):
self.name = name
self.lives = 3
```

Create an object

```
hero = Player("Mario")
print(hero.name)
```

Method

```
def lose_life(self):
self.lives -= 1
```

Mental model

A class is a blueprint. An object is one value created from that blueprint. self means this object.

Variables lists and input

Store information and receive user choices

Variables

```
player = "Mario"
lives = 3
is_ready = True
```

Input returns text

```
name = input("Your name: ")
print("Hello", name)
```

Lists

```
items = ["key", "star"]
items.append("mushroom")
first = items[0]
```

Useful list tools

len(items) counts items. value in items checks membership. items.remove(value) removes a matching item.

Index reminder

The first list index is 0. The last item can be read with items[-1].

Conditions and logic

Choose which code runs

Basic pattern

```
if coins >= 10:
print("1-UP")
elif coins >= 5:
print("Almost")
else:
print("Keep going")
```

Comparisons

== equal != not equal > greater < less >= at least <= at most

Logic

and needs both conditions. or needs either condition. not reverses True and False.

Common mistake

```
Use == to compare values. Use = to assign a value to a variable.
```


Future files and modules

Save data and use organized code

Read a text file

```
with open("notes.txt", "r") as file:
text = file.read()
```

Write a text file

with open("score.txt", "w") as file:
file.write(str(score))

Import a module

```
import random
choice = random.choice(items)
```

Main guard

```
if __name__ == "__main__":
main()
```

File habit

```
Use with so Python closes the file even if something goes wrong.
```

Future errors and exceptions

Handle expected problems

Read the traceback

Start at the final line for the error type and message, then check the named line in your code.

Common errors

SyntaxError: code shape is invalid. NameError: name is unknown. TypeError: operation does not fit the value type.

Expected input problem

```
try:
age = int(input("Age: "))
except ValueError:
print("Enter a whole number")
```

Do not hide every error

Catch errors you expect and can handle. Unexpected errors should stay visible while debugging.

For loops and range

Repeat a known number of times

Loop through a list

```
for enemy in ["Boo", "Goomba"]:
print(enemy)
```

What range means

range(4) produces 0, 1, 2, 3. It starts at 0 and stops before 4.

Three useful forms

```
range(4) -> 0,1,2,3
range(2, 5) -> 2,3,4
range(2, 9, 2) -> 2,4,6,8
```

Accumulator pattern

```
total = 0
for coin in [2, 4, 6]:
total = total + coin
```

While loops

Repeat while a condition stays True

Basic pattern

```
lives = 3
while lives > 0:
lives = lives - 1
```

Ask two questions

What condition keeps the loop running? What changes so that condition becomes False?

Avoid infinite loops

If the condition never becomes False, the loop never stops. Update the condition variable inside the loop.

Choose the loop

```
Use for for a list or known count. Use while when repetition depends
on a condition.
```

Useful controls

break exits a loop early. continue skips to the next iteration.

Future function patterns

Make code easier to reuse and test

Default argument

```
def greet(name, greeting="Hello"):
return f"{greeting}, {name}"
```

Small focused functions

A function should usually do one clear job. A good name describes that job.

Local variables

A variable created inside a function is normally local to that function.

Documentation

```
Place a short docstring under def to explain the inputs, behavior,
and returned value.
```

Future dictionaries and sets

Store labeled data and unique values

Dictionary

```
player = {"name": "Luigi", "lives": 3}
print(player["name"])
```

Safe lookup

`player.get("power", "small")` returns a default if the key is missing.

Loop through pairs

```
for key, value in player.items():
print(key, value)
```

Set

`unique_items = set(items)` removes duplicates. Sets are useful for membership checks.

Functions return and None

Give reusable code a clear name

Define and call

```
def power(base, bonus):
return base + bonus

score = power(7, 5)
```

Vocabulary

Parameters are names in the definition. Arguments are values sent into the call.

return

return sends a value back to the caller. Here, power(7, 5) returns 12.

None

None means no value. A function without return still runs, but its result is None.

Print is different

print displays information. return gives a value back so the program can reuse it.

Current course quick check

Use this before asking for help

Trace

Write the value of each variable after every line. For a loop, trace one iteration at a time.

Check the shape

`Look for missing colons, parentheses, quotes, indentation, or misspelled names.`

Test one piece

Run one expression, condition, loop, or function call before testing the whole program.

Explain

Can you say what each variable stores, when each loop stops, and what each function returns?

Search precisely

Try: Python add item to list, Python range start stop step, or Python function returns None.

Future strings and numbers

Convert and format information

Input conversion

```
age_text = input("Age: ")
age = int(age_text)
height = float(input("Height: "))
```

String methods

name.strip() removes edge spaces. name.lower() changes case. text.split(",") makes a list.

Formatted strings

```
coins = 12
message = f"You have {coins} coins"
```

Type check

type(value) reports a value's type. Common types are str, int, float, bool, list, and NoneType.